

agre-e

Architecture, Security & Step-by-Step Build Guide

Evidence-first private agreements for users, counsel and investors

Product architecture • data model • audit chain • security • implementation roadmap

Core product principle

**Private by default. Identifiable when necessary. Tamper-evident by design.
Explainable when challenged.**

1. Product definition

agre-e should be positioned as a private agreement evidence system, not merely a PDF signing or blockchain application. It helps parties create, identify, consent to, sign, preserve, verify and later explain the history of an agreement.

The system should distinguish integrity, identity, authentication, signing, evidentiary workflow and legal effect. A cryptographic hash proves data consistency; it does not by itself determine legal validity or admissibility.

2. Philosophies that should govern the product

Philosophy	How agre-e applies it
Zero Trust	Never trust a request merely because a user is logged in. Authenticate, authorize and record consequential actions.
Evidence by Design	Capture identity, exact document version, consent, signing events, timestamps, integrity proofs and the audit history as the transaction occurs.
Privacy by Design	Collect less, encrypt, segregate identity data, minimize public disclosure and control retention.
Defence in Depth	Use WAF/rate limits, authentication, authorization, validation, row-level security, encryption and audit logging in layers.
Least Privilege	Support, security, counsel and admin roles receive only the permissions needed for their function.
Append-only History	Never silently overwrite an executed document or evidentiary event. Amendments create new versions/events.
Secure by Default	Private object storage, deny-by-default authorization, server-side secrets and strong privileged-account authentication.

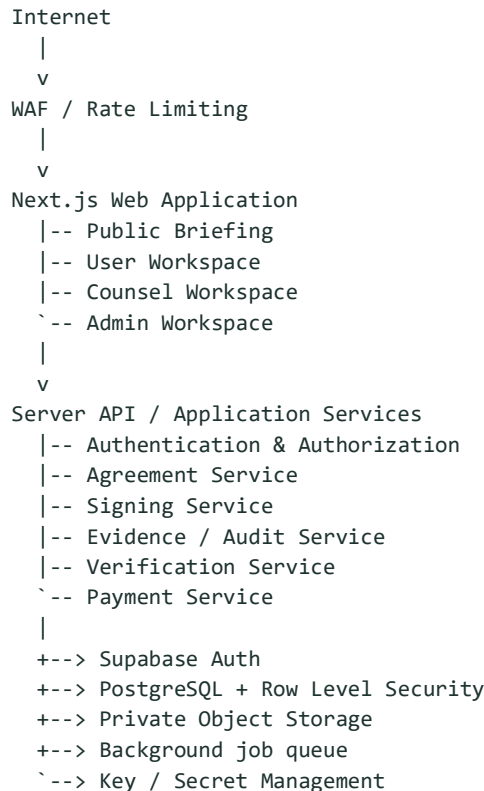
3. Legal and compliance design anchor

Kenya's Evidence Act contains provisions addressing electronic and digital evidence. Section 78A addresses factors including the reliability of generation/storage/communication, integrity maintenance and identification of the originator. Section 106B addresses admissibility conditions associated with computer output. These provisions should inform the evidence model, but Kenyan counsel should approve product claims and the certificate workflow before launch.

Kenya's Data Protection Act regulates personal-data processing and includes principles, data-subject rights, DPIA, retention, data protection by design/default, breach handling and international-transfer provisions. agre-e should therefore minimize personal data, define lawful processing, implement access controls, document retention and perform a DPIA before high-risk production use.

Important: the product should say what its technology verifies. Avoid UI claims such as "legally valid" or "court admissible". Prefer statements such as "document integrity verified" and "evidence chain verified."

4. Recommended system architecture



Start as a modular monolith. This keeps security boundaries explicit without the operational complexity of premature microservices. Split services later only where load, organizational ownership or independent security boundaries justify it.

5. Product surfaces and trust boundaries

Surface	Purpose	Critical boundary
Public	Marketing, briefing and minimal verification	Must never expose agreement contents or identity data.
User workspace	Create, invite, review, sign and export	Only agreements owned by/shared with the authenticated user.
Counsel workspace	Review shared matters/evidence and certificate preparation	Client/matter access must be explicitly granted.
Admin workspace	Operations, billing, security and support	Admin status must not automatically grant contract-reading rights.

Suggested hostnames: www.agre-e.com, app.agre-e.com, counsel.agre-e.com and admin.agre-e.com.

Separate surfaces can still share one codebase initially while enforcing distinct authorization policies.

6. Core data model

- profiles: application profile linked to the authentication identity.
- organizations / organization_members: optional business or law-firm tenancy.

- agreements: stable agreement identity, ownership, type and lifecycle state.
- agreement_versions: immutable versions with storage path and SHA-256 digest.
- agreement_parties: participants, role, signing requirement/order and invitations.
- identity_verifications: provider/status references; avoid needless copies of sensitive identity material.
- signature_requests / signature_events: request and completion events linked to the exact document version.
- consents: record the disclosures/terms accepted for the particular event.
- audit_events: append-only evidence chain.
- evidence_packages: generated manifest/document/event bundle metadata.
- evidence_certificates: counsel-reviewed certificate workflow state.
- payments: commercial transaction state, separate from evidentiary truth.
- security_events / admin_actions: privileged and security-sensitive activity.

7. SQL starter schema

```
create table agreements (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null references auth.users(id),
  title text not null,
  agreement_type text not null,
  status text not null check (status in
    ('draft','awaiting_parties','awaiting_signatures','completed','revoked')),
  created_at timestamptz not null default now(),
  completed_at timestamptz
);

create table agreement_versions (
  id uuid primary key default gen_random_uuid(),
  agreement_id uuid not null references agreements(id),
  version_number integer not null,
  storage_path text not null,
  sha256 text not null,
  created_by uuid not null references auth.users(id),
  created_at timestamptz not null default now(),
  unique(agreement_id, version_number)
);
```

Do not update the contents of an executed version. An amendment produces a new agreement_versions row and new audit events.

8. Authorization and Supabase RLS

Browser-side hiding is not authorization. Enforce permission again at the server/API and database. Enable Row Level Security for user-accessible tables and deny access unless an ownership, membership or explicit sharing policy permits it.

```
alter table agreements enable row level security;

create policy "party can read agreement"
on agreements for select
using (
  owner_id = auth.uid()
```

```

    or exists (
      select 1 from agreement_parties p
      where p.agreement_id = agreements.id
            and p.user_id = auth.uid()
    )
  );

```

Never expose `SUPABASE_SERVICE_ROLE_KEY` to browser code and never prefix it with `NEXT_PUBLIC_`. Keep privileged credentials only in server-side secret storage. Rotate privileged secrets if they are accidentally committed.

9. Evidence ledger and tamper-evident chain

The audit ledger should be append-only. Each event commits to the previous event hash. A change to an earlier canonical event then breaks verification of the subsequent chain. This is a tamper-evident chain and should not be marketed as blockchain.

```

E1 [agreement created]
  hash=A...
  |
  v
E2 [party verified] previous=A... hash=B...
  |
  v
E3 [party signed]   previous=B... hash=C...
  |
  v
E4 [stamp issued]   previous=C... hash=D...

create table audit_events (
  id uuid primary key default gen_random_uuid(),
  agreement_id uuid not null references agreements(id),
  actor_id uuid,
  event_type text not null,
  object_id uuid,
  occurred_at timestamptz not null default now(),
  ip_hash text,
  user_agent text,
  metadata jsonb not null default '{}',
  previous_hash text,
  event_hash text not null
);

```

Create event timestamps and hashes on the trusted server/database side. Define a deterministic canonical serialization before hashing so insignificant JSON formatting differences cannot cause inconsistent hashes.

10. Hashing implementation

```

import crypto from 'crypto';

export function sha256(data: string): string {
  return crypto.createHash('sha256')
    .update(data, 'utf8')
    .digest('hex');
}

```

SHA-256 answers an integrity question: “does this data produce the expected digest?” Authentication answers who accessed the application. A signing mechanism links an actor/action to a specific version. Keep those claims separate in both code and interface text.

11. Signing transaction

```
Invite -> authenticate -> authorize -> load exact version -> display agreement -> capture consent
-> confirm signing action -> write signature event -> append audit event -> evaluate completion
```

- The client submits intent; the server decides whether the action is allowed.
- Bind a signing event to `agreement_id` and `agreement_version_id`.
- Record the accepted disclosure/consent version.
- Use server-controlled timestamping for the event record.
- When all required participants have successfully completed required actions, transition the agreement to completed and freeze that executed version.

12. Definition of a Stamp

Treat “Stamp” as a product term with a precise technical meaning: a frozen agreement version plus completion of required party actions plus an integrity digest plus an evidence-chain record. Do not state that the stamp itself determines legal validity.

```
STAMP
Agreement: AG-9F2C8A
Version: 3
Required signers: 2 / 2
Completed: server timestamp
Document SHA-256: ...
Audit chain head/root: ...
Evidence package: EP-...
```

13. Evidence package

Generate a portable, machine-readable package after completion. The package should remain useful even if the visual application changes.

```
evidence-package.zip
  agreement.pdf
  manifest.json
  events.json
  verification.json
  signatures.json
  certificate-data.json

{
  "format": "agree-e-evidence-package",
  "version": "1.0",
  "agreement_id": "AG-123",
  "document_hash_algorithm": "SHA-256",
  "document_hash": "...",
  "event_chain_root": "...",
```

```
"created_at": "..."  
}
```

14. Verify chain workflow

```
Input reference/package  
-> recalculate document digest  
-> compare recorded digest  
-> reconstruct canonical events  
-> verify each previous_hash link  
-> verify event digests  
-> verify required signing/completion events  
-> return a precise technical result
```

Example user-facing result: “Document integrity verified. Evidence-chain integrity verified. Required signing events recorded. No broken chain links detected.” Avoid converting those technical findings into a legal conclusion.

15. 106B certificate workspace

Build this as a lawyer-reviewed workflow, not a “one-click legal validity certificate.” The workspace can assemble system identification, agreement/version identifiers, relevant timestamps, document digest, event timeline, storage/system information and export metadata. Kenyan counsel should determine the final declaration wording, responsible signatory and procedure.

```
Evidence package -> certificate workspace -> human legal review -> authorized attestation ->  
retained certificate record
```

16. Privacy architecture

- Keep identity attributes separated from agreement content where practical, joined by internal UUIDs.
- Do not put raw ID/passport numbers, agreement text, addresses or personal loan values on a public chain.
- Use private storage and short-lived authorized download URLs.
- Encrypt data in transit and at rest through the chosen platform capabilities; apply application-level protection to especially sensitive fields where the threat model justifies it.
- Define retention by data class: account, identity verification, draft, completed agreement, security log and payment record.
- Log privileged access to sensitive agreement/evidence material.
- Provide a documented path for applicable access, correction, deletion/restriction and portability requests after legal review.

17. Security baseline

Area	Minimum baseline
Authentication	Verified email; MFA for admin/counsel; session revocation; re-authentication for sensitive actions.
Authorization	Deny by default; RLS; server authorization;

Storage	ownership/membership checks; least privilege. Private buckets; expiring access; file size/type controls; malware-scanning workflow before trusted consumption.
Web	TLS; HSTS; CSP; secure cookies; appropriate CSRF protection; rate limiting; strict input/output handling.
Secrets	No secrets in client bundles or Git; environment/secret manager; rotation and access review.
Audit	Append-only security/admin/evidence events; UTC timestamps; restricted access; monitoring/alerting.
SDLC	OWASP ASVS security requirements; automated tests; SAST/dependency/secrets scanning; code review; penetration testing before high-risk launch.
Recovery	Encrypted backup strategy; restore tests; defined RPO/RTO; incident-response runbook.

18. Suggested project structure

```

agree-e/
  app/
    (public)/briefing/{counsel,user}
    (auth)/{login,register}
    app/{agreements,verify,evidence,settings}
    counsel/{matters,evidence,certificates}
    admin/{users,billing,security,audit}
  lib/
    auth/ authorization/ cryptography/ evidence/ agreements/ audit/ validation/
  supabase/
    migrations/ policies/ functions/
  tests/
    unit/ integration/ security/ evidence/

```

19. Step-by-step implementation plan

#	Build area	Done when
1	Repository & environments	Create Next.js + TypeScript, linting/tests, separate development/staging/production environments and server-side secret handling.
2	Authentication	Implement registration/login/email verification, session controls and MFA for privileged roles.
3	Database foundation	Create profiles, agreements, agreement_versions, parties and migration scripts. Enable RLS before application data flows.
4	Agreement lifecycle	Create draft, add parties, create immutable versions, archive/revoke; prevent silent

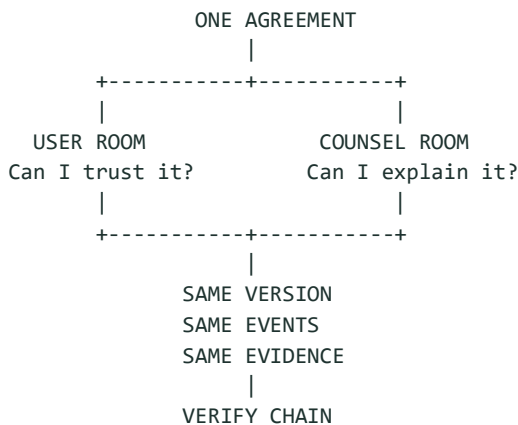
5	Authorization	mutation of executed versions. Centralize capability checks and add RLS tests for owner, party, counsel, support and admin cases.
6	Private storage	Upload generated/agreement files to private object paths; serve through authenticated/expiring authorization.
7	Signing	Bind sign/consent actions to an exact immutable version and persist server-side events.
8	Audit chain	Canonicalize event payloads, compute previous/event hashes server-side and prohibit ordinary update/delete access.
9	Stamp	Evaluate required completion conditions transactionally; freeze the executed version; append completion/stamp event.
10	Verification	Recalculate document and audit hashes independently; expose precise pass/fail reasons.
11	Evidence export	Create manifest, event history, integrity results and signed/attested package metadata as applicable.
12	Counsel workflow	Create controlled matter sharing, evidence timeline and certificate-draft workspace.
13	Admin/security	Separate operational, support and security permissions; log privileged actions.
14	Privacy/compliance	Complete DPIA, retention schedule, privacy notices, processor/vendor review and data-subject workflows with counsel.
15	Security testing	Run authorization/RLS tests, abuse tests, dependency/secrets scans, backup/restore tests and independent penetration testing.
16	Payments	Integrate payment only after evidence/signing state is stable; keep payment state separate from whether an evidentiary event occurred.

20. Critical tests before production

- IDOR/BOLA: User A cannot retrieve User B's agreement by changing a UUID.
- RLS: direct database/API requests cannot bypass UI restrictions.

- Version immutability: an executed agreement version cannot be edited through ordinary application paths.
- Hash verification: deliberately modify a copied package and confirm verification fails at the correct event/document.
- Replay/idempotency: duplicate sign requests do not create conflicting completion states.
- Race conditions: two parties signing simultaneously result in one consistent completion transition.
- Privilege boundaries: support/admin roles cannot read documents without the explicit privileged workflow.
- Secret scanning: no service-role/private credentials appear in the browser bundle or repository.
- Storage access: object URLs are not permanently public.
- Backup recovery: database and necessary evidence objects can be restored and verified.

21. Two-room investor briefing



Counsel room

- The problem: private digital agreements are easy to create, but their provenance and history may be difficult to reconstruct later.
- Show the Evidence Act design mapping without claiming the software decides admissibility.
- Demonstrate exact version, originator/identity events, integrity, event chain and evidence export.
- Show the certificate workspace as a counsel-designed workflow.
- Investment thesis: fund evidence infrastructure, privacy-preserving presentation and compliance/security engineering rather than “costume Ethereum.”

User-investor room

- Start with a familiar private agreement: loan, purchase or room/tenancy-type agreement.
- Verify identity at the appropriate assurance level; create/review the agreement; sign the exact version; complete the Stamp when all required parties finish.
- Demonstrate Verify chain and evidence export.
- Commercial hypothesis: charge for value-producing trust/evidence actions only after user testing. Do not present unvalidated pricing as forecast revenue.

22. Recommended product wording

Avoid	Prefer
“Legally verified”	“Document integrity verified”
“Court admissible”	“Evidence package verified; legal treatment is determined by the applicable process/court.”
“Blockchain secured”	“Tamper-evident cryptographic audit chain”
“Hash proves who signed”	“Hash verifies integrity; identity/signing records address actor/action evidence.”
“Your data is 100% secure”	“Private by default with layered technical and organizational controls.”

23. Sources and validation notes

Validate legal copy with Kenyan counsel before production. The product architecture deliberately separates technical evidence verification from legal conclusions.

- Kenya Law, Evidence Act (Cap. 80), including Part VII Electronic Records and sections 78A/106B: <https://new.kenyalaw.org/akn/ke/act/1963/46/eng@2023-12-11>
- Kenya Law, Data Protection Act (Cap. 411C): <https://new.kenyalaw.org/akn/ke/act/2019/24/eng@2022-12-31>
- Law Society of Kenya: current site describes membership as more than 20,000 practising advocates: <https://lsk.or.ke/>
- OWASP Application Security Verification Standard (ASVS): use as the application-security verification baseline.

24. Final product thesis

agre-e turns private agreements into verifiable evidence trails.

The user sees a simple agreement workflow. Counsel sees provenance and an evidence package. Security sees explicit trust boundaries and an append-only audit trail. Compliance sees controlled personal-data processing. Investors see an evidence and trust product rather than a blockchain pitch.